EMBL02E · IoT-Based Smart E-Waste Collection and Monitoring System

Web Pages and Workflows

Every page of the web app, what it shows, what you can do on it, and how the pages connect.

Date	29 October 2026
Written from	`docs/API_Design.md` (§7 web routes, §5 device API, §11 bin station API), `docs/System_Plan.md` (§4, §5.3–5.7, §6, §8.2–8.3), `docs/Database_Schema.md`, `docs/PROJECT-GUIDELINES.md` Step 8, and the web notes in `web/docs/` (`DESIGN.md`, `ACCOUNT-MANAGEMENT.md`, `CONTEXT.md`, `TRACK-A/B`, `resources/views/README.md`).
Companion	`web/docs/wireframes/all-pages-wireframe.html` — a wireframe of every page listed here, using the same page IDs.
Status	Working note for the web team. It restates the source docs and adds no rules. Where `docs/` and this note disagree, `docs/` wins.

Contents

1. How to read this document
2. Who uses the web app and what each role can open
3. Site map
4. Page inventory — every page on one table
5. Rules every page follows
6. Guest and account pages (G1–G5)
7. Signed-in pages for students and admins (S1–S9)
8. Admin pages (A1–A22)
9. System and error pages (E1–E5)
10. End-to-end workflows (W1–W16)
11. Appendix A — the bin's touchscreen (K1–K9)
12. Appendix B — the three emails
13. Appendix C — page-level decisions still open

1. How to read this document

- **Page IDs. G** = guest and account pages, **S** = signed-in pages (students and admins), **A** = admin pages, **E** = system and error pages, **K** = the bin's kiosk screens (not part of the web app). The wireframe file uses the same IDs.
- **Tags.** P28 marks a detail the API design proposes and the team has not confirmed (`API_Design.md` §9). Open marks a question with no answer in the docs (`API_Design.md` §10 or this note's Appendix C). Every web path and route name is itself proposal P28.
- **Suggested** marks a layout or display choice this note makes so the page can be drawn. It is not a rule; change it freely, as long as no new action, field or rule appears.
- **Example values** in this note and the wireframes only show shape. No point value, rate or cap is set by the plan.
- **Times** are stored in UTC and shown in Asia/Manila on every page.

At a glance

Group	Pages	Middleware	Shell (layout)
Guest and account (G)	5 pages + the email link handler	`guest`; G5 needs `auth`	`layouts/guest`
Signed-in (S)	8 pages + the `/` redirect	`auth`, `verified`	`layouts/app`
Admin (A)	22 pages	`auth`, `verified`, admin role	`layouts/admin`
System (E)	5 error pages	—	Laravel error views, styled
Bin kiosk (K)	9 screens	None — bin station on `localhost`	Bin station page, not Laravel

2. Who uses the web app and what each role can open

Visitor	Can open	Sent elsewhere when	Source
Guest (not signed in)	G1 Login, G2 Register, G3 Forgot password, G4 Set or reset password	Opens any signed-in page → redirected to `/login`	API §7.1, §7.2
Signed in, email not confirmed	G5 Confirm your email, the link in the email, the resend button, Log out	Opens any other signed-in page → redirected to `/email/verify` P48	API §7.2, §7.3
Student (confirmed)	S1–S9	Opens any `/admin` page → 403 (E1)	API §7.1, §7.3
Admin (confirmed)	A1–A22, and S1–S9 (admins have a QR token and appear on the user leaderboard) P31	—	API §7.3, §7.4
Inactive user (`status = inactive`)	Nothing signed-in. Can't sign in P29	Already signed in → loses access on the next request	API §7.2; ACCOUNT-MANAGEMENT
Unactivated user (admin-created, `password` NULL)	G3 and G4 only, to set a password	Login fails with the generic message	System Plan §4

Account words used on the pages

From `web/docs/CONTEXT.md`. Keep the two pairs apart in page copy.

Word	Means	Shown where
Activated / Not activated	A password has / has not been set	A4 user list, A6 edit user
Active / Inactive	Enabled / disabled by an admin (`users.status`)	A4, A6
Registration	A student creating their own account at `/register`	G2
Password setup	First password for an account an admin created	G4, A5, A6
Email confirmation	Following the link sent after registering	G5
Inactive organization	Closed to new students; existing members keep access	A7–A9, G2, A5, A6

Navigation (suggested)

Shell	Links
Guest	Log in · Register
Student	Home · Rewards · My redemptions · History · Leaderboard · Notifications · Profile · Log out
Admin	**Overview:** Dashboard · Notifications **Bins:** Bins · Session review **Deposits:** Transactions **Rewards:** Redemptions · Rewards · Points rate **People:** Users · Organizations **Reports:** Organization leaderboard · Reports **My account:** Student view (S2) · Log out

The design guide asks for visible, wrapping links on phones; the longer admin groups may fold with native `details` / `summary`. No scripted menu (DESIGN.md §5).

3. Site map

Guest and account

G1 Login `/login`

G2 Register `/register`

G3 Forgot password `/forgot-password`

G4 Set or reset password `/reset-password/{token}`

G5 Confirm your email `/email/verify`

Link handler `/email/verify/{id}/{hash}` (no page)

System

E1 403 Forbidden

E2 404 Not found

E3 419 Page expired

E4 429 Too many requests

E5 500 / 503 Server error

Signed in (student + admin)

S1 `/` → redirect by role

S2 Home `/home`

S3 Profile and QR `/profile`

S4 Deposit history `/transactions`

S5 Leaderboard `/leaderboard`

S6 Rewards `/rewards`

S7 Reward detail `/rewards/{reward}`

S8 My redemptions `/redemptions`

S9 Notifications `/notifications`

Admin `/admin`

Overview and reports

A1 Dashboard `/admin`

A2 Organization leaderboard

A3 Reports Open

People

A4 Users · A5 New user · A6 Edit user

A7 Organizations · A8 New · A9 Edit

Deposits

A10 Transactions · A11 Transaction detail

A12 Session review

Bins

A13 Bins · A14 Register bin · A15 Bin detail

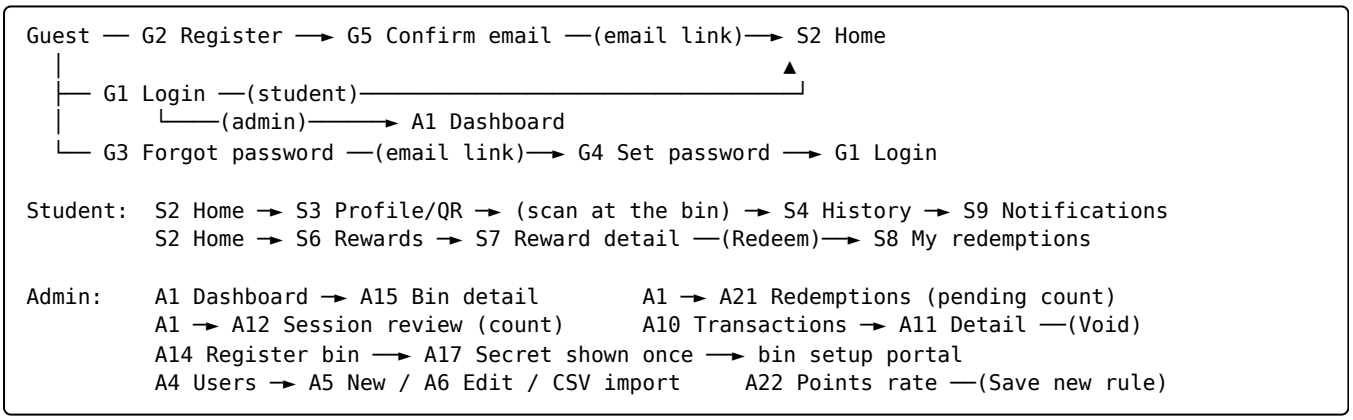
A16 Edit bin · A17 Bin secret (shown once)

Rewards

A18 Rewards · A19 New · A20 Edit

A21 Redemptions · A22 Points rate

Main paths through the site



4. Page inventory

Every page the web app renders. Form posts that have no page of their own are listed with the page they belong to. View paths follow `web/resources/views/README.md`. All paths and names: P28.

ID	Page	GET path	Route name	Form posts on this page	Module	View
G1	Log in	`/login`	`login`	`POST /login`	Identity	`identity/auth/login`
G2	Register	`/register`	`register`	`POST /register`	Identity	`identity/auth/register`
G3	Forgot password	`/forgot-password`	`password.request`	`POST /forgot-password` (`password.email`)	Identity	`identity/auth/forgot-password`
G4	Set or reset password	`/reset-password/{token}`	`password.reset`	`POST /reset-password` (`password.update`)	Identity	`identity/auth/reset-password`
G5	Confirm your email	`/email/verify`	`verification.notice`	`POST /email/verification-notification` (`verification.send`), `POST /logout`	Identity	`identity/auth/verify-email`
—	Email link (no page)	`/email/verify/{id}/{hash}`	`verification.verify`	—	Identity	Redirects to S2
S1	Root redirect (no page)	`/`	—	—	Identity	→ S2 or A1
S2	Home	`/home`	`home`	`POST /logout` (every shell)	Rewards	`rewards/home`
S3	Profile and QR	`/profile`	`profile.show`	`POST /profile/qr` (`profile.qr.regenerate`)	Identity	`identity/profile/show`
S4	Deposit history	`/transactions`	`transactions.index`	—	Deposits	`deposits/transactions/index`
S5	Leaderboard	`/leaderboard`	`leaderboard`	—	Reporting	`reporting/leaderboard`
S6	Rewards	`/rewards`	`rewards.index`	—	Rewards	`rewards/rewards/index`
S7	Reward detail	`/rewards/{reward}`	`rewards.show`	`POST /redemptions` (`redemptions.store`)	Rewards	`rewards/rewards/show`
S8	My redemptions	`/redemptions`	`redemptions.index`	—	Rewards	`rewards/redemptions/index`
S9	Notifications	`/notifications`	`notifications.index`	`POST /notifications/{notification}/read`	Notifications	`notifications/index`
A1	Dashboard	`/admin`	`admin.dashboard`	Polls `GET /admin/dashboard/summary` (JSON)	Reporting	`reporting/admin/dashboard`

A2	Organization leaderboard	`/admin/leaderboard/organizations`	`admin.leaderboard.organizations`	—	Reporting	`reporting/admin/organization-leaderboard`
A3	Reports	`/admin/reports`	`admin.reports`	**Open**	Reporting	`reporting/admin/reports`
A4	Users	`/admin/users`	`admin.users.index`	`POST /admin/users/import`, `POST /admin/users/{user}/password-link`	Identity	`identity/admin/users/index`
A5	New user	`/admin/users/create`	`admin.users.create`	`POST /admin/users`	Identity	`identity/admin/users/create`
A6	Edit user	`/admin/users/{user}/edit`	`admin.users.edit`	`PUT /admin/users/{user}`, `PUT …/password`, `POST …/password-link`	Identity	`identity/admin/users/edit`
A7	Organizations	`/admin/organizations`	`admin.organizations.index`	—	Identity	`identity/admin/organizations/index`
A8	New organization	`/admin/organizations/create`	`admin.organizations.create`	`POST /admin/organizations`	Identity	`…/organizations/create`
A9	Edit organization	`/admin/organizations/{organization}/edit`	`admin.organizations.edit`	`PUT /admin/organizations/{organization}`	Identity	`…/organizations/edit`
A10	Transactions	`/admin/transactions`	`admin.transactions.index`	—	Deposits	`deposits/admin/transactions/index`
A11	Transaction detail	`/admin/transactions/{transaction}`	`admin.transactions.show`	`POST …/{transaction}/void`	Deposits	`deposits/admin/transactions/show`
A12	Session review	`/admin/sessions/review`	`admin.sessions.review`	None defined **Open**	Deposits	`deposits/admin/sessions/review`
A13	Bins	`/admin/bins`	`admin.bins.index`	—	Devices	`devices/admin/bins/index`
A14	Register bin	`/admin/bins/create`	`admin.bins.create`	`POST /admin/bins`	Devices	`devices/admin/bins/create`
A15	Bin detail	`/admin/bins/{device}`	`admin.bins.show`	`POST /admin/bins/{device}/secret` (suggested placement)	Devices	`devices/admin/bins/show`
A16	Edit bin	`/admin/bins/{device}/edit`	`admin.bins.edit`	`PUT /admin/bins/{device}`	Devices	`devices/admin/bins/edit`
A17	Bin secret (once)	Response to `POST /admin/bins` or `…/secret`	—	—	Devices	`devices/admin/bins/secret`

A18	Rewards (admin)	`/admin/rewards`	`admin.rewards.index`	—	Rewards	`rewards/admin/rewards/index`
A19	New reward	`/admin/rewards/create`	`admin.rewards.create`	`POST /admin/rewards`	Rewards	`…/rewards/create`
A20	Edit reward	`/admin/rewards/{reward}/edit`	`admin.rewards.edit`	`PUT /admin/rewards/{reward}`	Rewards	`…/rewards/edit`
A21	Redemptions (admin)	`/admin/redemptions`	`admin.redemptions.index`	`POST …/{redemption}/fulfil`, `POST …/{redemption}/cancel`	Rewards	`rewards/admin/redemptions/index`
A22	Points rate	`/admin/points-rate`	`admin.points-rate.show`	`POST /admin/points-rate`	Rewards	`rewards/admin/points-rate/show`

Not in the list on purpose: delete pages (there are no delete routes, P33), self-service profile or password editing (deferred, `API_Design.md` §10 Q3 and ACCOUNT-MANAGEMENT), and the registration approval pages from the earlier prototype (superseded, ACCESS-FLOW-PLAN).

5. Rules every page follows

Topic	Rule	Source
Rendering	Server-rendered Blade with Tailwind. No AJAX CRUD, no chart library, no frontend framework. The only script is the admin dashboard's summary poll.	System Plan §3; AGENTS §4.3
Forms	Plain `method="POST"` with `@csrf`. `PUT` routes use `@method('PUT')`.	API §7.1
After a form	Success → redirect to a page with a flash message (shown inline, no auto-dismiss). Validation failure → back to the form with errors and old input; password and secret fields are never refilled.	API §7.1; DESIGN §6, §8
Errors on a form	An error summary near the heading, linked to fields, plus a message under each invalid field.	DESIGN §6
Lists	Every list is paginated. Wide tables scroll inside their own labelled region, never the whole page.	AGENTS §5.1; DESIGN §5
Times	Stored UTC, shown in Asia/Manila, with the zone named in table headers where dates are compared.	AGENTS §2.9; DESIGN §7
Units	Always visible: `184.6 g`, `240 pts`, `17.5 %`. The browser never calculates points.	DESIGN §4
Own data only	A user sees only their own transactions, redemptions and notifications.	API §7.1
No deletes	Users, organizations, rewards and bins are switched off with `status` or `is_active`.	API §7.4 P33
Status labels	Show the documented state as text (`COMPLETED` → "Completed"). Colour only supports the label.	DESIGN §3, §6
Missing data	A bin with no reading shows **No reading**, never 0 %.	DESIGN §7; API §7.5
Balance vs points earned	Balance = sum of all ledger rows (can be negative after a void). Leaderboards use *points earned* = EARN minus transaction REVERSALs; redeeming never lowers a rank. Label them differently.	System Plan §5.6; Schema §9
Mobile	Every page works on a phone (single column below 40 rem).	Guidelines Step 8; DESIGN §5

6. Guest and account pages

G1 Log in

`GET/POST /login · login`

Purpose Sign in with email and password. One form for students and admins; there is no role choice.

Shows Email, Password, **Log in** button. Links: Forgot password (G3), Register (G2). A flash after a password reset or logout.

Workflow

1. User enters email and password, presses **Log in**.
2. Success: the session is regenerated, `users.last_login_at` is set P6, then admin → A1 `/admin`, student → S2 `/home` P29. An unconfirmed student lands on G5 instead.
3. Failure: back to G1 with the email kept and one message.

Rules

- Wrong password, an account with no password (not activated) and an inactive account all show **the same message** P29.
- 5 attempts a minute per email and IP address pair; the 6th shows a "too many attempts, try again in N seconds" message P29.

Sources API §7.2; TRACK-A A3; ACCOUNT-MANAGEMENT

G2 Register

`GET/POST /register · register`

Purpose A student creates their own account. No admin approval.

Fields

Field	Rule
First name, Last name	Required, at most 100 characters each
Student number	Required P46, at most 50, unique
Email	Required, valid, at most 255, unique
Organization	Required. A select of organizations whose `is_active` is true
Password, Confirm password	Required, at least 8 characters, must match P41

Workflow

1. Student fills the form, presses **Create account**.
2. Server inserts the `users` row (`student`, `active`, `email_verified_at` NULL) and its QR credential in one transaction.
3. After the commit the confirmation email is sent (Laravel `Registered` event). The student is signed in and redirected to G5.
4. If the email fails to send, the account still stands and G5 says to use the resend button.

Rules Always creates a student, never an admin. 5 attempts a minute per IP P46 (over the limit → E4). On a validation error the form returns with errors and old input, password fields empty.

Sources API §7.2, §7.6 "Register"; System Plan §4; TRACK-A A4

G3 Forgot password

`GET/POST /forgot-password · password.request / password.email`

Purpose	Ask for a link to set a new password. Also the way an admin-created student gets a fresh set-password link after the first one expired.
Shows	Email field, **Email me a link** button, link back to G1.
Workflow	1. User enters an email and submits. 2. The page always shows the same acknowledgement, whether or not the email has an account P30. 3. An active account gets the reset email (Appendix B); an inactive or unknown one gets nothing.
Rules	Resending a link makes earlier links unusable; links last 60 minutes (ACCOUNT-MANAGEMENT; Laravel `auth.passwords.*.expire`; API §10 Q4).
Sources	API §7.2; ACCOUNT-MANAGEMENT "Inactive accounts and password links"

G4 Set or reset password

`GET /reset-password/{token} · POST /reset-password`

Purpose	One form for both emails: "set your password" (admin-created account) and "forgot password".
Shows	Email (prefilled from the link), New password, Confirm password, **Save password**. Hidden token.
Workflow	1. User opens the link from the email. 2. Enters and confirms a password (at least 8 characters). 3. Success → G1 with "Your password has been saved. Log in." The account is now *activated*.
States	Expired, used or superseded link → the form returns "This link is no longer valid" with a link to G3. Inactive account → same generic refusal.
Rules	Keep the route name `password.reset`; Laravel's email builds its link from it. A password change ends the account's other sessions on their next request.
Sources	API §7.2; ACCOUNT-MANAGEMENT

G5 Confirm your email

`GET /email/verify · verification.notice`

Purpose	The only page a signed-in student with an unconfirmed email can see.
Shows	"We sent a link to *{email}*. Open it to finish setting up your account." **Send the link again** button. **Log out** button. A flash when a link was resent, or a warning when the first send failed.
Workflow	1. Student opens the link in the email (`/email/verify/{id}/{hash}`). 2. Laravel checks the signed link, sets `email_verified_at` and redirects to S2. 3. If the link expired (60 min default), the student presses **Send the link again**.
Rules	Resend is limited to 6 a minute P46. Every other signed-in page redirects here until confirmed, including the QR page, so a student can't deposit first P48. A bad or expired link → E1 (403) with a link back to G5.
Sources	API §7.2, §7.3; TRACK-A A1, A4

7. Signed-in pages (students and admins)

Open to every signed-in user whose email is confirmed, admins included P31. Guests go to G1; unconfirmed users go to G5. **S1** (`GET /`) is a redirect only: admin → A1, student → S2.

S2 Home

`GET /home · home · Rewards`

Purpose	Points balance and the latest deposits. Login destination for students.
Shows	• **Available points** — the balance, `SUM(point_ledger.points_delta)`. The page's focal panel. May be negative after a void. • Actions: **View QR code** (S3), **Browse rewards** (S6). • **Recent transactions** — the latest few rows of S4 (date, weight, points, status) and a "See all" link. How many rows is not set (suggested: 5). • Empty state: "No deposits yet. Show your QR code at the bin to earn points."
Workflow	Read-only. After a deposit at the bin, the new transaction and the higher balance appear here on the next page load.
Sources	API §7.3; Guidelines Step 8 "Current points"; DESIGN §5 "Student home"

S3 Profile and QR code

`GET /profile · POST /profile/qr`

Purpose	The student's details, their organization, and the QR code they show at the bin.
Shows	• **Profile:** name, email, student number, role, account since. Read-only (self-editing is deferred). • **Organization:** name, code, description (Guidelines "Organization information"). Admins have none: "No organization (admin account)". • **QR code:** `user_qr_credentials.token` rendered on the server as SVG, black on white, with its quiet zone P32. "Last changed" = the row's `updated_at`. • Help: "Hold this code up to the scanner on the bin. It identifies you for one deposit; it can't spend points."
Workflow	**Regenerate QR code:** the student presses **Make a new QR code** (secondary button). Beside it: "Your old code stops working at once." The server overwrites the token and redirects back with "Your QR code has been replaced."
Rules	Only shown once the email is confirmed P48. Never recolour, crop or overlay the code (DESIGN §7). Token is 40 random letters and digits P35.
Sources	API §7.3; System Plan §5.5 "QR code"; Schema §3; TRACK-A A7

S4 Deposit history

`GET /transactions · transactions.index · Deposits`

Purpose	The student's own deposits ("Disposal/transaction history").
Shows	Paginated table, newest first: Date (Asia/Manila), Transaction code (`TXN-YYYYMMDD-XXXXXX` P25), Bin, Weight (g, the session's `net_weight_g`), Points, Status (**Completed** or **Voided**). A voided row shows "Points reversed" and the date.
Rules	Only the user's own rows (through `deposit_sessions.user_id`). Whether rejected, failed, expired or cancelled *sessions* (no transaction) are listed is not stated Open; the wireframe lists transactions only.
Sources	API §7.3; Schema §6, §8

S5 Leaderboard

`GET /leaderboard · leaderboard · Reporting`

Purpose	Rank every user by points earned.
Shows	Paginated table: Rank, Name, Organization (blank for admins), **Points earned**. A note: "Ranked by points earned from deposits. Redeeming rewards does not lower your rank."
Rules	Points earned = sum of `EARN` rows minus the `REVERSAL` rows that link to a transaction. Students and admins both appear. Never show the spendable balance here.
Sources	API §7.3; System Plan §5.6

S6 Rewards

`GET /rewards · rewards.index · Rewards`

Purpose	Catalogue of rewards the user can redeem ("Available rewards").
Shows	The user's balance (suggested, for context). Cards or rows for rewards whose `is_active` is true: name, short description, points cost, stock ("12 left" / "Out of stock"), and a **View reward** link to S7. Paginated.
Empty	"No rewards are available right now."
Sources	API §7.3; Schema §10; DESIGN §6 "rewards"

S7 Reward detail and redeem

`GET /rewards/{reward} · POST /redemptions`

Purpose	One reward, with the redeem form ("Reward redemption").
Shows	Name, full description, points cost each, stock, the user's current balance. Form: **Quantity** (whole number, 1 or more), a line showing "Total: cost × quantity" from the server's cost, **Redeem reward** button.
Workflow	1. User sets the quantity and presses **Redeem reward**. 2. `RedemptionService`, in one transaction: lock the user row, sum the ledger, refuse if the balance is below cost × quantity, decrement stock with `WHERE stock_quantity >= ?`, insert the redemption (`PENDING`, `RDM-…` code) and the negative `REDEEM` ledger row. 3. Success → S8 with "Redeemed {reward}. Collect it from the admin office once it is marked fulfilled." (destination suggested). 4. Refusal → back to S7 with the reason; nothing changes.
Refusals	Not enough points · Not enough stock · Reward no longer available (inactive or missing) · Quantity not a whole number ≥ 1.
Unavailable	When the balance is below one unit's cost or stock is 0, keep the button disabled with the reason beside it (DESIGN §6). The server still checks.
Sources	API §7.3, §7.6 "Redeem a reward"; System Plan §6.1

S8 My redemptions

`GET /redemptions · redemptions.index · Rewards`

Purpose	The user's own redemptions and their state.
Shows	Paginated table, newest first: Date, Redemption code, Reward, Quantity, Points spent, Status — **Pending** (waiting for the admin), **Fulfilled** (with date), **Cancelled** ("Points returned").
Sources	API §7.3; Schema §11

S9 Notifications

`GET /notifications · POST /notifications/{notification}/read`

Purpose In-app messages. Students and admins use the same page.

Shows Paginated list, newest first: title, message, time, and **Unread**/**Read** as a text label (unread also in bold). Each unread item has a **Mark as read** button.

Types

Type P40	Who gets it	Raised when
`DEPOSIT_COMPLETED`	The depositing user	A deposit's `complete` commits
`REDEMPTION_FULFILLED`	The redeeming user	An admin fulfils the redemption (A21)
`BIN_FULL`	Every admin	A reading turns FULL after one that was not
`BIN_FAULT` P43	Every admin	A reading carries a new fault code. An open service panel raises none

Workflow Press **Mark as read** → `read_at` is set → back to S9 with the item shown as Read.

Sources API §7.3, §7.7; Schema §12

8. Admin pages

All under `/admin` with `auth`, `verified` and an admin role check; route names start with `admin.`. A student gets E1 (403).

Overview and reports

A1 Dashboard

`GET /admin · GET /admin/dashboard/summary (JSON)`

Purpose	Live view of the bin and of work waiting for an admin. Login destination for admins.
Shows	1. **Needs attention** (first): pending redemptions (count → A21), sessions awaiting review (count → A12), any bin that is FULL, has a fault, or has its panel open. 2. **Bins** table: name (→ A15), fill % with a bordered meter, status **OK**/**Full**, service panel **Open**/**Closed**, fault (e.g. `ACTUATOR_JAM`, shown with a plain label), last seen. Every non-retired bin. "No reading" when the bin has none. 3. **Today** band (since midnight Asia/Manila): deposits, weight collected (g), points awarded. 4. "Updated {time} (Asia/Manila)".
Workflow	The one script polls the summary every 10 seconds P34 and updates the numbers in place. If a poll fails, the page keeps the last numbers, labels them "last known", keeps the last update time and shows **Could not refresh** in a polite live region. It never shows zeros or says the bin is offline because the poll failed.
Summary fields	`bins[]` (id, name, fill_percent, status, panel_open, fault_code, last_seen_at), `today.deposits`, `today.weight_g`, `today.points_awarded`, `pending_redemptions`, `sessions_awaiting_review`, `generated_at` (UTC, converted by the script).
Sources	API §7.4, §7.5; DESIGN §7; Guidelines "Dashboard", "Bin-level monitoring"

A2 Organization leaderboard

`GET /admin/leaderboard/organizations`

Shows	Paginated table: Rank, Organization, Code, **Points earned** (total for the students currently in it).
Rules	Students only: filter `role = 'student'` before grouping, so no blank "admin" group appears. Same metric as S5.
Sources	API §7.4; System Plan §5.6; Guidelines "Organization/college leaderboard"

A3 Reports

`GET /admin/reports · admin.reports`

Status	Open The plan names a reports page and scheduled reports but not their contents (API §10 Q2). The wireframe shows an empty shell with the heading and a note. Don't build fields until the team decides.
Sources	API §7.4, §10; DESIGN §7

People

A4 Users

`GET /admin/users · POST /admin/users/import · POST /admin/users/{user}/password-link`

Shows	• **New user** button (→ A5). • Paginated table: Name (→ A6), Email, Student number, Role, Organization, Status (**Active**/**Inactive**), Activated (**Yes**/**Not activated**), Last login. Row action for a not-

activated account: **Send set-password email** (or **Resend**).

- **Import students from CSV** panel: file picker, the column list (`student_number, first_name, last_name, email, organization_code`, with a header row), **Import** button.

Workflow: import

1. Admin chooses a CSV and presses **Import**.
2. The whole file is validated. Any bad row → nothing is imported; the page lists each row number and its errors.
3. All good → every row becomes a confirmed `student` with a QR credential and no password; flash "Imported N students. Send each one a set-password email from the list."
4. No email is sent by the import P36. A bulk "send to all" is deferred (API §10 Q6).

Workflow: send link Press **Send set-password email** → the link is sent during the request; earlier links stop working. Success flash, or "The email could not be sent. Set a password for this user on their edit page instead."

Rules An organization code must match an existing, active organization. Mail is limited to 100 a day on the current plan (System Plan §7.2), so large imports are invited over several days or given passwords by an admin.

Sources API §7.4, §7.6 "CSV import"; System Plan §4, §7.2; ACCOUNT-MANAGEMENT

A5 New user

`GET /admin/users/create · POST /admin/users`

Fields First name, Last name (required, ≤100) · Email (required, unique, ≤255) · Student number (optional, unique, ≤50) · Role (`student` / `admin`) · Organization (required for a student, empty for an admin; active organizations only) · Status (`active` / `inactive`).

Workflow

1. Admin fills the form, presses **Create user**.
2. One transaction inserts the user (password NULL, email counted as confirmed P47) and the QR credential.
3. The set-password email is sent during the request.
4. → A4 (or A6) with "User created. A set-password email was sent." or, if the send failed, "User created, but the email was not sent."

Sources API §7.6 "Create or edit a user"

A6 Edit user

`GET /admin/users/{user}/edit · PUT /admin/users/{user} · PUT …/password · POST …/password-link`

Shows Three separate forms on one page:

1. **Details** — the A5 fields, **Save changes**.
2. **Password email** — current state (Activated / Not activated), **Send set-password email**.
3. **Set a password** — fallback when email can't be sent: Password, Confirm password (≥ 8), **Set password**. Beside it: "Tell the user this password in person. They can change it later with Forgot password."

Also read-only: email confirmed at, last login, created.

Rules

- The last active, activated admin can't be set inactive or changed to student — including by themself. The form returns "At least one other active admin with a password must remain."
- A student can't be newly assigned to an inactive organization; an existing membership may stay when other details are edited.

- Setting a user inactive ends their access on their next request. A password, email or role change ends their other sessions and revokes outstanding password links.

Sources API §7.4, §7.6; ACCOUNT-MANAGEMENT "Prevent admin lockout", "Inactive organizations", "Sessions after account changes"

A7–A9 Organizations · New · Edit

`/admin/organizations[/create | /{organization}/edit]`

A7 list Paginated table: Name (→ A9), Code, Description, Active (**Active**/**Inactive**). **New organization** button.

A8 / A9 form Name (required, ≤150) · Code (required, unique, ≤50; used by the CSV import) · Description (optional, ≤255) · Active (checkbox). **Create organization** / **Save changes** → A7 with a flash.

Rules No delete. Setting an organization inactive keeps its students and their access, removes it from the register form and blocks new assignments, including CSV import.

Sources API §7.4, §7.6; CONTEXT; ACCOUNT-MANAGEMENT

Deposits

A10 Transactions

`GET /admin/transactions · admin.transactions.index`

Shows

- **Total collected weight** (focal number): sum of `net_weight_g` over deposits whose transaction is `COMPLETED`; voided deposits excluded. Shown in g (and kg).
- Paginated table: Date, Code (→ A11), User, Organization, Bin, Weight (g), Points, Status (**Completed**/**Voided**).

Sources API §7.4; System Plan §5.6; Guidelines "E-waste transaction records", "Total collected weight"

A11 Transaction detail and void

`GET /admin/transactions/{transaction} · POST …/void`

Shows

- **Transaction:** code, status, completed at, points awarded, reward rule used (name and rate).
- **Deposit session:** session id, user (and organization), bin, opened at, closed at.
- **Weight:** net weight (g), stable (yes/no), samples.
- **Verification:** method (`cam_diff`), passed, score (0–1).
- **Actuator:** OK (yes/no), cycle time (ms).
- If voided: reason, voided by, voided at.

Workflow: void

1. Only while **Completed**: the **Void this transaction** form shows a required **Reason** (≤ 500) and a destructive-style button, with "The student loses {points} pts. Their balance may go below zero."
2. Server locks the row, confirms it is still COMPLETED, sets VOIDED with reason, by and at, and `PointsService` writes a `REVERSAL`.
3. → A11 with "Transaction voided." A second void is impossible (unique ledger constraint); the form is gone on a voided transaction.

Rules The voided deposit still counts toward that day's cap and is excluded from collected weight.

Sources API §7.4, §7.6 "Void a transaction"; System Plan §5.7, §6.1

A12 Session review

`GET /admin/sessions/review · admin.sessions.review`

Purpose	Lists `ACCEPTED` sessions older than the review threshold (10 minutes): the item may be in the bin but `complete` never arrived.
Shows	Paginated table: Session id, User, Bin, Opened at, Accepted for (age), Weight (g), Verification score. Empty state: "No sessions are waiting for review."
Actions	None defined Open (API §10 Q1). The list is read-only. A late `complete` from the bin still credits the student and removes the row.
Sources	API §4.1, §7.4; System Plan §5.3 rule 6, §6.3

Bins

A13 Bins

`GET /admin/bins · admin.bins.index`

Shows	**Register a bin** button (→ A14). Paginated table: Name (→ A15), Device code, Location, Status (**Active**/**Inactive**/**Retired**), Fill % with meter, OK/Full, Panel, Fault, Last seen, Firmware.
Sources	API §7.4; Guidelines "Bin-level monitoring"

A14 / A16 Register bin · Edit bin

`GET /admin/bins/create · POST /admin/bins · GET …/{device}/edit · PUT …/{device}`

Fields	Device code (required, unique, ≤50; the bin's `X-Device-Code`) · Name (required, ≤100) · Location (optional, ≤255) · Empty distance (mm, required, above 0) · Fill threshold (%, 0–100, default 90) · Load cell offset and scale factor (optional; saving new values sets *calibrated at*) · Status (`active` / `inactive` / `retired`).
Workflow	A14: **Register bin** → the server generates the secret → A17. A16: **Save changes** → A15 with a flash. The bin picks up new thresholds and calibration on its next `GET /device/config`.
Rules	Only an `active` bin is accepted by the device API; `inactive` / `retired` get `DEVICE_INACTIVE`. A retired bin leaves the dashboard.
Sources	API §5.2, §7.4, §7.6 "Register or edit a bin"; Schema §4

A15 Bin detail

`GET /admin/bins/{device} · POST /admin/bins/{device}/secret`

Shows	• **Now:** fill % and meter, OK/Full, service panel open/closed, current fault with a plain explanation, last seen, firmware version. • **Setup:** device code, location, status, empty distance, fill threshold, calibration values and calibrated at. **Edit bin** (→ A16). • **Fill history:** paginated `bin_telemetry` rows, newest first: Time, Distance (mm), Fill %, Status, Panel, Fault. • **Device secret: Issue a new secret** with "The old secret stops working at once. The bin can't deposit until you enter the new one in its setup portal."
Faults	`ACTUATOR_JAM`, `ACTUATOR_TIMEOUT`, `ACTUATOR_RETRACT_FAILED` (latched until the panel is opened and closed); `STATION_UNREACHABLE`, `CAMERA_UNAVAILABLE`, `SCANNER_UNAVAILABLE` (clear themselves). The server only shows them; the bin decides them.
Sources	API §5.3, §7.4; Schema §5; System Plan §8.2

A17 Bin secret (shown once)

`Response to POST /admin/bins or POST /admin/bins/{device}/secret`

Shows	The device code and the new secret (64 hex characters) in a monospace box, with: "Copy this now. It is shown only once. Enter it in the bin's setup portal with the device code." Link: **Done — go to the bin** (→ A15).
Rules	Never shown again, never refilled into a form, never in a URL. Lost secret → issue a new one P39.
Sources	API §7.6; System Plan §8

Rewards

A18–A20 Rewards · New · Edit

`/admin/rewards[/create | /{reward}/edit]`

A18 list	Paginated table: Name (→ A20), Points cost, Stock, Active. **New reward** button.
A19 / A20 form	Name (required, ≤150) · Description (optional) · Points cost (whole number ≥ 1) · Stock (whole number ≥ 0) · Active (checkbox). Help on pricing: "Decide how many grams should earn the cheapest reward; its cost = grams × points per gram." Changing a price never changes past redemptions.
Sources	API §7.4, §7.6; System Plan §5.7 "Pricing rewards"

A21 Redemptions

`GET /admin/redemptions · POST …/{redemption}/fulfil · POST …/{redemption}/cancel`

Shows	Paginated table, pending first (suggested): Date, Code, User, Organization, Reward, Quantity, Points, Status, Actions. Pending rows have **Mark fulfilled** (primary) and **Cancel** (destructive).
Workflow	• **Fulfil:** admin hands over the item, presses **Mark fulfilled** → status FULFILLED, `fulfilled_at` set; the student gets a `REDEMPTION_FULFILLED` notification. • **Cancel:** status CANCELLED, stock returned, a `REVERSAL` gives the points back. Beside the button: "Returns {points} pts and the stock." • Both lock the row and check it is still PENDING; the loser of a race gets "This redemption was already {fulfilled/cancelled}."
Sources	API §7.4, §7.6 "Fulfil or cancel"; Guidelines "Redemption management"

A22 Points rate

`GET /admin/points-rate · POST /admin/points-rate`

Shows	• **Current rule:** name, points per gram, minimum weight (g), maximum weight (g or "No limit"), daily points cap (or "No cap"), in effect since. • **No rule** state: a warning "No points rule is in effect. The bin refuses every deposit (`NO_REWARD_RULE`) until you save one." • **Save a new rule** form: Name, Points per gram (> 0), Minimum weight (≥ 1 ÷ points per gram), Maximum weight (optional; above minimum, at most the hardware limit), Daily cap (optional, whole ≥ 1). **Save new rule**. • **Past rules** table: name, the four values, from, to.
Workflow	Saving closes the current rule (`effective_to` = now) and starts the new one at once, in one transaction P38. Sessions already open keep the rule they opened under. No future-dated rules.
Rules	Values need instructor approval (guidelines Step 9). The hardware limit is a config setting P37 whose value is still open (API §10 Q7).

Sources	API §7.4, §7.6 "Save a points rule"; System Plan §5.7

9. System and error pages

Laravel renders these for web routes. Style them with the guest shell; each has a heading, one plain sentence and a way back. Device routes never get these pages — they always answer JSON (AGENTS §2.8).

ID	Status	When a user sees it	Way back
E1	403 Forbidden	A student opens an `/admin` page; an email confirmation link is bad or expired	Home (by role); "Send a new link" for the email case
E2	404 Not found	A record id that doesn't exist or isn't the user's	Home
E3	419 Page expired	A form posted after the session or CSRF token expired	"Go back and try again" / Log in
E4	429 Too many requests	Register over 5 a minute, resend over 6 a minute (login shows its limit on G1 instead)	"Wait a minute and try again"
E5	500 / 503	Server error or database down	Retry; no success state is shown

10. End-to-end workflows

Each workflow lists the pages it passes through (IDs from Section 4) and what happens on the server and the bin in between.

W1 · Student registers and confirms their email

Student · G2 → G5 → email → S2

1. G2: fills name, student number, email, organization, password → **Create account**.
2. User and QR credential are created; the student is signed in; the confirmation email is sent after the commit.
3. G5: "Check your email." Every other signed-in page redirects here.
4. Student opens the link → `email_verified_at` set → S2 Home.

Other paths

Email failed to send or link expired → **Send the link again** on G5 (6/min). Duplicate email or student number → G2 with errors. Daily email limit reached → resend the next day.

W2 · Sign in and sign out

Any user · G1 → S2 or A1

1. G1: email + password. Success → admin to A1, student to S2; unconfirmed → G5.
2. Wrong password, no password or inactive → the same message on G1. Six tries in a minute → wait message.
3. **Log out** (every shell) → session ended, CSRF token regenerated → G1 with "You have logged out."

W3 · Admin creates accounts and they set a password

Admin, then student · A5 or A4 import → email → G4 → G1

1. Admin creates one user on A5 (email sent at once) or imports a CSV on A4 (no email).
2. For imported users, admin presses **Send set-password email** on each A4 row.
3. Student opens the link → G4 → sets a password → G1 → signs in. The account is now activated and already confirmed.

Other paths

Email can't be sent → admin uses **Set a password** on A6 and tells the student. Link expired → student uses G3 Forgot password, or admin resends (only the latest link works).

W4 · Forgot password

Any user · G1 → G3 → email → G4 → G1

1. G1 → **Forgot password?** → G3, enter email → generic acknowledgement.
2. Active account receives the link (60 min) → G4 → new password → G1. Other sessions end.

W5 · Deposit e-waste at the bin and see the points

Student at the bin · S3 → bin (K1–K9) → S2 / S4 / S9

1. Student opens S3 and holds the QR code to the bin's scanner while the screen shows **K1 Ready** (scanner armed only when the bin is ready).
2. Controller calls `POST /device/sessions`. Server checks, in order: valid QR, user active, bin not full, a rule in effect, under the daily cap, no open session on another bin. Pass → session `OPEN` (90 s) → **K2 Welcome {first name}**.
3. Student places the item → **K3 Weighing**. Controller waits for a stable weight, lights the platform, asks the bin station's camera to compare with the empty reference.
4. Controller calls `deposit` with weight and camera result. Server checks verification, stability, weight above 0, minimum, maximum. Pass → `ACCEPTED` and a points preview → **K4 Accepted: {g}, {pts}**. Fail → `REJECTED` → **K7 Problem** with a plain reason.
5. Actuator pushes the item into the bin; the controller judges the stroke from its current.
6. Controller calls `complete`. Actuator OK → transaction + `EARN` ledger row in one DB transaction → **K5 Done: +{pts}, balance {n}**; `DEPOSIT_COMPLETED` notification. Actuator failed → session `FAILED`, no points.
7. Student later sees the new row on S2 and S4, a higher balance, and the notification on S9.

Other paths

Cancel on the screen or 90 s timeout → session cancelled/expired, no points. Server unreachable after the item went in → **K6 Points pending**; the bin retries `complete` before the next deposit, and credits arrive later. Not credited within 10 min → listed on A12. Bin full → **K8 Bin full**. Daily cap reached → K7 "You have reached today's limit".

W6 · Redeem a reward and collect it

Student, then admin · S6 → S7 → S8 … A21 → S9

1. S6 → **View reward** → S7 → choose quantity → **Redeem reward**.
2. Server locks the user, checks balance, decrements stock, writes the redemption (PENDING) and the REDEEM row → S8 with a flash.
3. A1 shows the pending count → admin opens A21, hands over the item, presses **Mark fulfilled**.
4. Student gets `REDEMPTION_FULFILLED` on S9; S8 shows Fulfilled.

Other paths

Not enough points or stock → S7 with the reason. Admin cancels a pending redemption → stock and points returned; S8 shows Cancelled.

W7 · Replace a lost or shared QR code

Student · S3

1. S3 → **Make a new QR code** → the token is overwritten; the old one fails at the bin at once (`INVALID_QR`).

W8 · First-time setup of a new install

Installer and first admin · seed → G3/G4 → A1 → A14 → A17 → A22

1. Operator runs migrations and seeds the first admin (no password). With `MAIL_MAILER=log`, they request a reset on G3 and open the logged link → G4 → set the password.
2. Admin signs in → A1. Creates organizations (A8) so students can register.
3. A14 **Register bin** → A17 shows the secret once → enter device code, secret, server URL and station URL in the bin's setup portal.
4. A22 saves the first points rule (values approved by the instructor).
5. Only now does the bin accept deposits. Before it is registered its calls get `INVALID_SIGNATURE`; before a rule exists every scan gets `NO_REWARD_RULE`.

W9 · Monitor the bin and respond to alerts

Admin · A1 → S9 → A15 → A16

1. The bin sends telemetry at boot, after each deposit, on any panel or fault change, and on an interval. The server computes fill % and OK/FULL.
2. A1 updates every 10 s. A reading that turns FULL raises `BIN_FULL`; a new fault raises `BIN_FAULT` — one notification per admin on S9.
3. Admin opens A15 for the fill history and fault, empties or fixes the bin. A latched actuator fault clears only after the service panel is opened and closed.
4. If the empty distance or calibration changed, admin updates them on A16.

W10 · Void a wrong deposit

Admin · A10 → A11

1. A10 → open the transaction → A11, check the weight, camera score and actuator result.
2. Enter a reason → **Void this transaction** → status VOIDED + REVERSAL row. Student's balance and points earned drop; the balance may go negative.
3. The deposit disappears from total collected weight but still counts toward that day's cap.

W11 · Change the points rate

Admin · A22

1. A22 → fill the new rule → **Save new rule**. The old rule ends now; the new one applies to sessions opened from now on.
2. The bin shows new weight limits after its next config fetch; the server always judges with the session's own rule.

W12 · Manage rewards and stock

Admin · A18 → A19/A20

1. A19 creates a reward with cost and stock; A20 changes price, stock, or switches it off (inactive rewards leave S6).

W13 · Deactivate an account or an organization

Admin · A6, A9

1. A6 → Status **Inactive** → the user is signed out on their next request, can't sign in, can't get password links, and their QR is refused at the bin (`USER_INACTIVE`).
2. The last active, activated admin can't be deactivated or demoted.
3. A9 → organization inactive → hidden from G2 and new assignments; existing students keep access.

W14 · Replace a bin's secret

Admin · A15 → A17

1. A15 → **Issue a new secret** → A17 shows it once. The old secret stops working at once.
2. Enter it in the bin's setup portal; the bin's next signed call succeeds.

W15 · Review stuck sessions

Admin · A1 → A12

1. A1 shows "sessions awaiting review" > 0 → A12 lists ACCEPTED sessions older than 10 minutes.
2. What the admin does next is **Open** (API §10 Q1). A late `complete` still credits normally.

W16 · Check standings

Anyone signed in · S5; admin · A2

1. S5 ranks every user by points earned. A2 ranks organizations by their students' points earned.

Appendix A — the bin's touchscreen (K1–K9)

Not part of the web app. The bin station (Python on the Pi) serves this page on `localhost`, shown by Chromium in kiosk mode on the 7" 1024×600 touchscreen. The controller picks the screen with `PUT /station/v1/screen`; the page never talks to Laravel. All of this is P44. The website design guide does not govern it (DESIGN §1), but it is listed here because the deposit workflow (W5) runs through it.

ID	Screen	Values	Shown when	LED
K1	`IDLE` — Ready	—	Bin ready, scanner armed: "Scan your QR code to start"	Green
K2	`WELCOME`	`name`	Session opened: "Hi {name}. Place your item on the platform." Cancel button	Amber
K3	`WEIGHING`	—	Weighing and camera check. Cancel button	Amber
K4	`ACCEPTED`	`weight_g`, `points`	Deposit accepted; actuator moving	Amber
K5	`DONE`	`points`, `balance`	`complete` succeeded	Green
K6	`PENDING`	—	Item went in but `complete` couldn't reach the server: "Your points are on their way"	Amber
K7	`PROBLEM`	`message`	Any refusal or fault, in plain words, never a code	Red
K8	`BIN_FULL`	—	Server reports the bin full	Red
K9	`OFFLINE`	—	Server unreachable, or the controller silent past the station's timeout	Red

Plain messages for K7 (suggested wording)

Cause (code)	Message on the screen
`INVALID_QR`	We don't recognise this QR code. Open your profile to get your current code.
`USER_INACTIVE`	This account can't deposit right now. Please contact the admin office.
`DAILY_LIMIT_REACHED`	You've reached today's points limit. Come back tomorrow.
`NO_REWARD_RULE`	The bin isn't taking deposits yet. Please try again later.
`DUPLICATE_TRANSACTION`	You already have a deposit in progress on another bin.
`VERIFICATION_FAILED` / `NO_ITEM`	We couldn't see an item on the platform. Scan again to retry.
`WEIGHT_UNSTABLE`	The item kept moving while we weighed it. Scan again to retry.
`ZERO_WEIGHT`	This item is too light to earn points.
`OVER_MAX_WEIGHT`	This item is too heavy for the bin. Please take it to the admin office.
`SESSION_EXPIRED`	Time ran out. Scan again to start over.
Panel open / actuator fault / station fault	This bin is out of service. Please try again later.
Actuator failed (`FAILED`)	The item couldn't be moved into the bin. No points were added. Please tell the admin office.

Appendix B — the three emails

Only three emails exist; every other message is in-app on S9 (System Plan §7.2). Mail is sent during the request (no queue). A failed send never fails the action.

Email	Sent when	Link goes to	Lifetime	If it fails
Confirm your email	A student registers (G2); resend on G5	`/email/verify/{id}/{hash}` → S2	60 min (`verification.expire`)	G5 says to use the resend button
Set your password	Admin creates a user (A5) or presses Send on A4/A6	G4 `/reset-password/{token}`	60 min	Admin is told; uses Set a password on A6
Reset your password	G3 for an active account	G4 `/reset-password/{token}`	60 min	User sees the same generic acknowledgement

Current delivery is Resend on the project domain (100 a day, 3,000 a month); after handover, the university's SMTP. Development uses `MAIL_MAILER=log`.

Appendix C — page-level decisions still open

Do not build these into pages until the team or the docs owner decides. Raise them as their own decision, not inside a page PR.

#	Question	Affects	Source
1	What can an admin do with a session in review?	A12, W15	API §10 Q1
2	What do reports and scheduled reports contain?	A3	API §10 Q2
3	Can a signed-in user change their own details or password? (Deferred for the account milestone.)	S3	API §10 Q3; ACCOUNT-MANAGEMENT
4	Is 60 minutes long enough for the set-password email?	G4, Appendix B	API §10 Q4
5	Bulk "send set-password emails" after a CSV import? (Deferred.)	A4	API §10 Q6
6	The hardware weight limit value, used when saving a rule	A22	API §10 Q7
7	Which QR library renders the SVG?	S3	API §10 Q8
8	Does S4 list deposits that earned nothing (rejected, failed, cancelled, expired sessions), or only transactions?	S4	Not in the docs
9	How many rows does "recent transactions" on S2 show?	S2	Not in the docs
10	Should list pages have filters or search (users, transactions, redemptions)? None is designed.	A4, A10, A21	Not in the docs
11	Unread notification count in the navigation? Not designed; would need a query on every page.	All shells	Not in the docs
12	Where does a successful redemption redirect (S8 assumed)?	S7	API §7.1 says "a page"

All web paths, route names and placements remain proposal P28 until the team confirms `API_Design.md` §9.